

Verteilte Systeme – TaskGrid+

Dokumentation

von

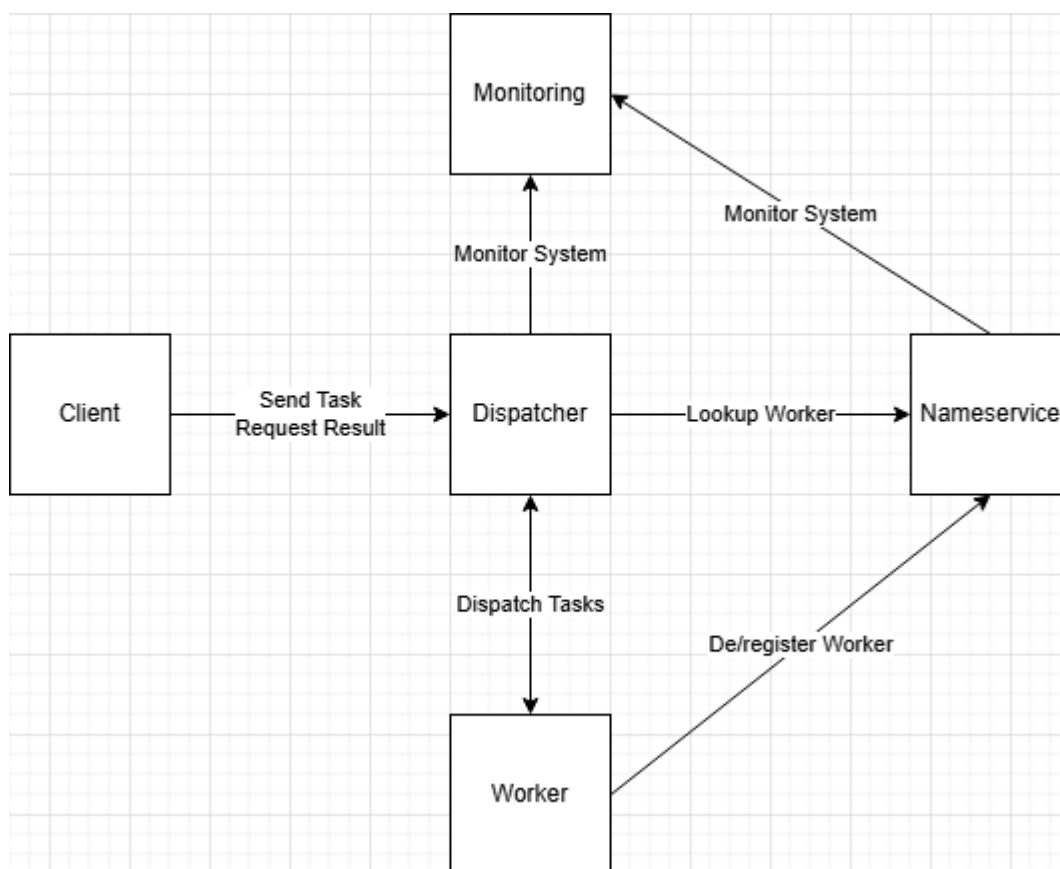
Mathias Druica
David Fischer
Linus Marschall
Milan Kiele

Inhalt

1	Architekturdiagramm	3
2	Begründung der Architektur	3
3	Kommunikationsprotokoll	5
4	Ablaufdiagramm	9
5	Beschreibung der Verwendeten Externen Bibliotheken und deren Funktionalität	10
6	Dokumentation der Schnittstellen	12
7	Dokumentation des Buildprozesses	21
8	Startanleitung	22
9	Auflistung eingesetzter Technologien	23
10	Testprotokoll mit Logs	24

Dies ist die Dokumentation der Software „TaskGrid+“. Sie stellt ein verteiltes System dar, welches simple Aufgaben bearbeiten kann. Dabei kann ein Client eine Aufgabe an das System übergeben, dieses verteilt im Hintergrund automatisiert die Aufgabe an einen freien Worker, welcher diese bearbeitet und zurück an den Client schickt.

1 Architekturdiagramm



2 Begründung der Architektur

Das oben dargestellte Architekturdiagramm zeigt den Aufbau des verteilten Systems. Jede Komponente ist als eigenständige Python-Klasse implementiert und läuft isoliert in einem eigenen Docker-Container. Eine zentrale Eigenschaft des Systems ist seine

verteilte Struktur. Um dieser Anforderung gerecht zu werden, wurde ein modularer Architekturansatz gewählt, bei dem jede Komponente eine klar definierte Aufgabe übernimmt. Dadurch wird das System **skalierbar, wartbar und flexibel erweiterbar**.

Funktionale Aufteilung der Komponenten

- **Client:** Stellt Aufgabenanfragen an das System und fragt die Ergebnisse der Verarbeitung ab. Der Client agiert als Schnittstelle zwischen Nutzer und System, ohne Einblick in interne Abläufe.
- **Dispatcher:** Vermittelt Aufgaben vom Client an passende Worker. Er übernimmt das Routing der Anfragen und entkoppelt die Kommunikation zwischen Client und ausführenden Einheiten.
- **Worker:** Verarbeiten die ihnen zugewiesenen Aufgaben. Sie können dynamisch zum System hinzugefügt oder entfernt werden.
- **Nameservice:** Verwaltet die Registrierung und Deregistrierung der Worker. Zudem stellt er dem Dispatcher Informationen zur Verfügung, um verfügbare Worker aufzulösen.
- **Monitoring:** Überwacht das gesamte System. Es sammelt Informationen darüber, welche Worker aktiv sind und welche Aufgaben sie ausführen.

Vorteile des modularen Designs

Der modulare Aufbau erlaubt es, beliebig viele Worker zu betreiben und dynamisch in das System zu integrieren. Neue Worker können sich beim Nameservice registrieren, wodurch sie automatisch in die Verteilung durch den Dispatcher aufgenommen werden. Das Monitoring-Modul sorgt dafür, dass jederzeit Transparenz über die Systemstruktur und den Zustand der einzelnen Worker besteht. So wird sichergestellt, dass etwaige Fehler oder Engpässe frühzeitig erkannt werden.

Ein weiteres wesentliches Prinzip dieser Architektur ist die Abstraktion der Verteilung gegenüber dem Nutzer. Der Client kommuniziert ausschließlich mit dem Dispatcher, nicht direkt mit den Workern. Dadurch bleibt die zugrunde liegende verteilte Verarbeitung für den Benutzer vollständig verborgen. Das System verhält sich aus

Sicht des Nutzers wie eine zentrale, monolithische Anwendung – trotz seiner internen Dezentralität.

Erweiterbarkeit und Zukunftssicherheit

Dank der Entkopplung der Komponenten ist das System leicht erweiterbar. Weitere Funktionseinheiten wie Caching, Persistenzdienste oder Sicherheitsmechanismen (z. B. Authentifizierung, Zugriffskontrolle) können problemlos ergänzt werden. Auch ein skalierbarer Dispatcher-Cluster ist vorstellbar, sollte das Anfragevolumen steigen.

3 Kommunikationsprotokoll

Im Folgenden werden das verwendete Kommunikationsprotokoll RPC (spezifisch gRPC), die Struktur einer Task-Nachricht, die entsprechenden Schnittstellen, das Error-Handling und die Docker-Netzwerkconfiguration beschrieben.

Kommunikationsprotokoll: gRPC

Das TaskGrid-System verwendet gRPC für die Kommunikation zwischen Komponenten. gRPC ermöglicht eine typensichere, bidirektionale Kommunikation über HTTP/2 und ist in der Protobuf-Datei taskgrid.proto definiert. Die Hauptschnittstellen sind:

- ClientService (Port 50052, Dispatcher): Ermöglicht Clients, Aufgaben zu senden (SendTask), Ergebnisse abzufragen (RequestResult) und Statistiken zu erhalten (GetTaskStats).
- WorkerService (dynamische Ports, z.B. 50060-50068, Worker): Verarbeitet Aufgaben (ProcessTask) und registriert Worker beim NameService.
- NameService (Port 50051): Verwaltet Worker-Registrierung (RegisterWorker), -Abfrage (LookupWorker), -Deregistrierung (DeregisterWorker) und Statistiken (GetWorkerStats).

- **MonitoringService** (Port 50053): Liefert Systemstatistiken (**GetSystemStats**) und bietet eine REST-API auf Port 8080.

Task-Message

Die zentrale Datenstruktur ist die Task-Nachricht, definiert in `taskgrid.proto`:

```
message Task {  
  
    int32 id = 1;           // Eindeutige Task-ID  
  
    string type = 2;        // Aufgabentyp (z.B. reverse, sum)  
  
    string payload = 3;     // Eingabedaten der Aufgabe  
  
    string result = 4;      // Ergebnis der Aufgabe  
  
    string status = 5;      // Status (PENDING, COMPLETED, FAILED)  
  
    int64 timestamp_created = 6; // Erstellungszeitpunkt  
  
    int64 timestamp_completed = 7; // Abschlusszeitpunkt  
  
}
```

Diese Struktur wird von allen Komponenten verwendet, um Aufgaben zu erstellen, zu verarbeiten und deren Status zu verfolgen.

Schnittstellen

Die Schnittstellen sind in `taskgrid.proto` definiert und in `taskgrid_pb2_grpc.py` implementiert. Die wichtigsten Methoden sind:

- **ClientService.SendTask**: Client sendet Aufgabe an Dispatcher, erhält Task-ID.
- **ClientService.RequestResult**: Client fragt Ergebnis einer Aufgabe ab.
- **WorkerService.ProcessTask**: Worker verarbeitet Aufgabe und gibt Ergebnis zurück.

- `NameService.RegisterWorker`: Worker registriert sich mit Typ und Adresse.
- `MonitoringService.GetSystemStats`: Liefert Systemstatistiken (z.B. aktive Worker, ausstehende Aufgaben).

Die Kommunikation ist synchron (unary-unary), wobei jede Anfrage eine einzelne Antwort erwartet.

Die genauere Schnittstellenbeschreibung findet sich in einem späteren Abschnitt.

Error-Handling

Für das Error-Handling verwenden wir die in gRPC integrierten Mechanismen:

- gRPC Status Codes: Fehler werden durch Statuscodes wie `UNAVAILABLE` (Service nicht erreichbar), `NOT_FOUND` (Worker nicht verfügbar) oder `INVALID_ARGUMENT` (ungültige Eingabe) signalisiert. Der Dispatcher markiert fehlgeschlagene Aufgaben als `FAILED` und speichert Fehlerinformationen im Task-Objekt.
- Timeouts und Retries: Clients und Dispatcher verwenden konfigurierbare Timeouts (z.B. 5 Sekunden) und bis zu 3 Wiederholungsversuche bei transienten Fehlern (z.B. Netzwerkunterbrechungen). Dies wird durch gRPC-Client-Bibliotheken (wie `grpc.Retry`) implementiert.
- Fallback-Mechanismus: Falls kein Worker für einen Aufgabentyp verfügbar ist, speichert der Dispatcher die Aufgabe in der Warteschlange und versucht später erneut, einen Worker zu finden.
- Logging: Fehler werden zentral im Monitoring-Service protokolliert, inklusive Statuscode, Fehlermeldung und Zeitstempel.

Docker-Netzwerk

Die Docker-Konfiguration (docker-compose.yml) definiert ein dediziertes Bridge-Netzwerk (benannt taskgrid) für die Kommunikation:

- NameService: Läuft auf Port 50051, zentrale Registrierungsstelle für Worker.
- Dispatcher: Läuft auf Port 50052, kommuniziert mit NameService (NAMESERVICE_ADDRESS=nameservice:50051).
- Worker: Unterschiedliche Typen (z.B. reverse, sum) auf Ports 50060-50068, registrieren sich beim NameService.
- Monitoring: Läuft auf Ports 50053 (gRPC) und 8080 (REST), überwacht Dispatcher und NameService.
- Test-Client: Kommuniziert mit Dispatcher (DISPATCHER_ADDRESS=dispatcher:50052).

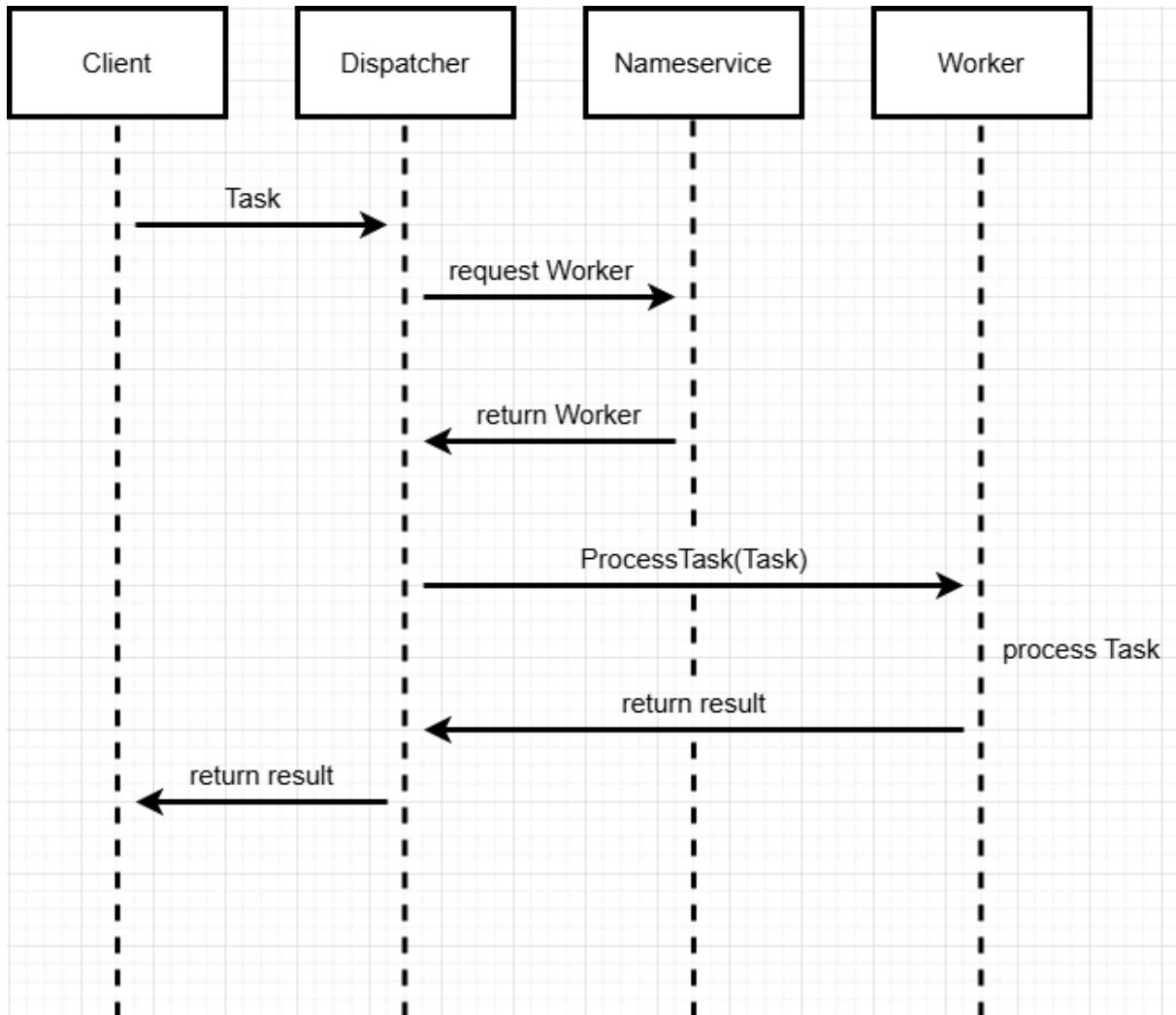
Alle Services sind im taskgrid-Netzwerk verbunden, wobei DNS-Namen (z.B. nameservice, dispatcher) für die service-to-service-Kommunikation verwendet werden. Die depends_on-Direktiven stellen sicher, dass NameService vor Dispatcher und Workern startet, was wichtig ist, da sich Worker beim NameService anmelden müssen, bevor der Dispatcher auf diese zugreifen kann. Ports sind extern zugänglich (z.B. 50051, 50052) über Forwarding, um Client-Interaktionen zu ermöglichen.

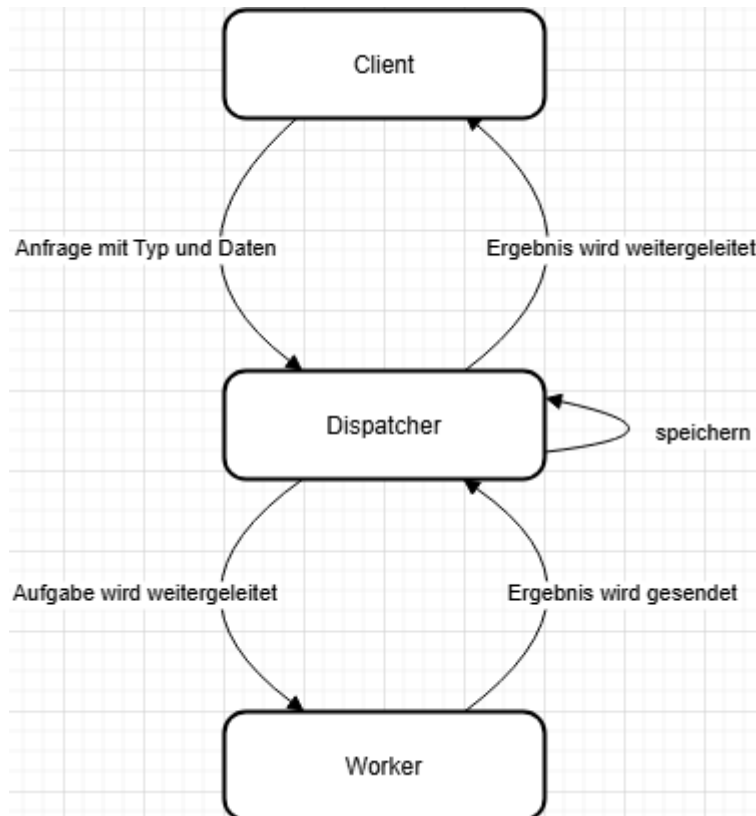
Fazit des Kommunikationsprotokolls

Das Kommunikationsprotokoll des TaskGrid-Systems basiert auf gRPC und bietet eine robuste, typensichere Kommunikation mit klar definierten Schnittstellen. Die Task-Nachricht ermöglicht eine einheitliche Datenverarbeitung, während Error-Handling durch gRPC-Statuscodes, Timeouts und Retries abgesichert ist. Das Docker-Netzwerk sorgt für eine zuverlässige service-to-service-Kommunikation und sorgt für Skalierbarkeit.

4 Ablaufdiagramm

Diese beiden Diagramme zeigen den Ablauf der Anfrage, Bearbeitung und Rückgabe eines Tasks. Dabei wird ausgehend von der Anfrage eines Clients, durch den Dispatcher ein Worker beim Nameservice angefragt, welcher dann den Task des Clients bearbeitet. Das Ergebnis wird dann dem Client zurückgegeben.





5 Beschreibung der Verwendeten Externen Bibliotheken und deren Funktionalität

Zur Realisierung der Aufgabenstellung haben wir uns entschieden, die folgenden externen Bibliotheken zu verwenden. Sie dienen hauptsächlich der Realisierung der RPC-Kommunikation.

grpcio

Die Bibliothek grpcio stellt die Python-Implementierung von gRPC (Google Remote Procedure Call) bereit. gRPC ist ein Framework zur Kommunikation zwischen verteilten Systemen über Netzwerkgrenzen hinweg. Es basiert auf HTTP/2 und nutzt Protocol Buffers zur effizienten Serialisierung von Daten.

Funktion im Projekt: Ermöglicht den Aufruf von Remote Procedures zwischen den Systemkomponenten (z. B. Client, Dispatcher, Worker, Nameservice).

grpcio-tools

Dieses Paket erweitert grpcio um Werkzeuge zum Kompilieren von .proto-Dateien. Dabei werden automatisch Python-Klassen generiert, die gRPC-Server und -Client-Stubs sowie Nachrichtenklassen enthalten.

Funktion im Projekt: Wird verwendet, um aus einer Protokolldatei (.proto) die entsprechenden Python-Klassen für gRPC-Nachrichten und Schnittstellen zu erzeugen.

protobuf

protobuf ist die offizielle Python-Bibliothek für Protocol Buffers von Googles, zur serialisierten Datenbeschreibung. Diese wird für die effiziente und strukturierte Datenübertragung genutzt.

Funktion im Projekt: Definiert die Nachrichtenformate und Strukturen, die über gRPC zwischen den Komponenten ausgetauscht werden.

python-dotenv

python-dotenv erlaubt das Einlesen von Konfigurationswerten aus einer .env-Datei. Diese Werte werden in die Umgebungsvariablen geladen und können z. B. API-Schlüssel, Ports oder Pfade enthalten.

Funktion im Projekt: Ermöglicht eine einfache und sichere Konfiguration von Umgebungsvariablen für verschiedene Komponenten, wie die Portnummern für gRPC-Server.

cryptography

Die Bibliothek cryptography bietet Funktionen für Verschlüsselung, Signaturen und Hashing Operationen. Sie basiert auf OpenSSL und ist eine der am weitesten

verbreiteten Krypto-Bibliotheken in Python.

Funktion im Projekt: Wird verwendet, um sichere Kommunikation mittels gRPC zu gewährleisten.

6 Dokumentation der Schnittstellen

Die vorliegende Schnittstellenbeschreibung dokumentiert die Kommunikationsschnittstellen des TaskGrid-Systems. Das System besteht aus mehreren Komponenten: Client, Dispatcher, NameService, Worker und Monitoring. Die Kommunikation zwischen diesen Komponenten erfolgt primär über gRPC, während die Monitoring-Komponente zusätzlich eine REST-API bereitstellt. Diese Dokumentation beschreibt die Schnittstellen, ihre Funktionen und die zugrunde liegende Kommunikationslogik.

Ports

Das TaskGrid-System ist modular aufgebaut, wobei die Komponenten mittels gRPC über folgende Ports kommunizieren:

- Nameservice: Port 50051
- Dispatcher: Port 50052
- Monitoring: Port 50053

Zusätzlich bietet die Monitoring-Komponente eine REST-API auf Port 8080.

gRPC-Schnittstellen

Die Kommunikation im TaskGrid-System basiert auf gRPC. Die Protokolldefinitionen sind in einer Protobuf-Datei (taskgrid.proto) definiert. Diese Datei ermöglicht die automatisierte Code-Generation der erforderlichen RPC-Funktionen, welche anschließend von den verschiedenen Komponenten verwendet werden. Dadurch wird eine einheitliche Implementierung dieser Funktionen ermöglicht, welche hierdurch aufeinander abgestimmt sind.

Im Folgenden werden die Schnittstellen der einzelnen Komponenten beschrieben.

Client

Der Client hat seine Schnittstellen zum Dispatcher, an den er verschiedene Anfragen/Tasks stellt und hinterher die Ergebnisse der Tasks abfragt.

- SendTask
 - Request: SendTaskRequest(type: string, payload: string)
 - Response: SendTaskResponse(task_id: int32)
 - Beschreibung: Sendet eine Aufgabe mit einem bestimmten Typ und Payload an den Dispatcher. Der Dispatcher weist der Aufgabe eine eindeutige ID zu und fügt sie in die Warteschlange ein. Der Client erhält die Task-ID zurück.

Beispiel:

```
response = stub.SendTask(taskgrid_pb2.SendTaskRequest(  
    type="reverse",  
    payload="Hello, World!"  
))
```

- GetResult
 - Request: GetResultRequest(task_id: int32)
 - Response: GetResultResponse(task: Task)
 - Beschreibung: Fragt das Ergebnis einer Aufgabe anhand ihrer Task-ID ab. Der Dispatcher gibt ein Task-Objekt zurück, das den Status (PENDING, COMPLETED, FAILED) und das Ergebnis enthält.

Beispiel:

```
response = stub.GetResult(taskgrid_pb2.GetResultRequest(task_id=1))
```

Dispatcher

Der Dispatcher hat verschiedene Schnittstellen, um Aufgaben an Worker zu verteilen, und fragt die Ergebnisse der Worker ab. Die Protokolldefinitionen sind in einer Protobuf-Datei (taskgrid.proto) definiert. DispatchTasks verteilt die Tasks über den NameService an verschiedene Worker über SendTask und mit RequestResult wird anschließend das Ergebnis angefragt. GetTaskStats dient der Monitoring-Komponente und stellt Statistiken über die derzeit ausführenden Tasks bereit.

- DispatchTasks
 - Request: DispatchTasksRequest(tasks: repeated Task)
 - Response: DispatchTasksResponse(success: bool, dispatched_tasks: repeated int32)
 - Beschreibung: Verteilt mehrere Aufgaben an verfügbare Worker basierend auf ihren Typen. Der Dispatcher fragt die Worker-Adressen vom NameService ab und sendet die Aufgaben an die entsprechenden Worker. Gibt die IDs der erfolgreich verteilten Aufgaben zurück.

Beispiel:

```
response = stub.DispatchTasks(taskgrid_pb2.DispatchTasksRequest(  
    tasks=[  
  
        taskgrid_pb2.Task(id=1, type="reverse", payload="Hello"),  
  
        taskgrid_pb2.Task(id=2, type="sum", payload="1,2,3")  
  
    ]  
  
))
```

- SendTask
 - Request: SendTaskRequest(type: string, payload: string)
 - Response: SendTaskResponse(task_id: int32)
 - Beschreibung: Empfängt eine Aufgabe mit einem bestimmten Typ und Payload vom Client. Der Dispatcher weist der Aufgabe eine eindeutige

ID zu und fügt sie in die Warteschlange ein. Die Task-ID wird an den Client zurückgegeben.

Beispiel:

```
response = stub.SendTask(taskgrid_pb2.SendTaskRequest(  
  
    type="reverse",  
  
    payload="Hello, World!"  
  
))
```

- **GetTaskStats**

- Request: TaskStatsRequest()
- Response: TaskStatsResponse(pending_tasks: int32, task_stats: map<string, TaskTimingStats>)
- Beschreibung: Liefert Statistiken über die Aufgaben, einschließlich der Anzahl ausstehender Aufgaben und detaillierter Zeitstatistiken pro Aufgabentyp.

Beispiel:

```
response = stub.GetTaskStats(taskgrid_pb2.TaskStatsRequest())
```

- **RequestResult**

- Request: RequestResultRequest(task_id: int32)
- Response: RequestResultResponse(task: Task)
- Beschreibung: Fragt das Ergebnis einer Aufgabe anhand ihrer Task-ID ab. Der Dispatcher gibt ein Task-Objekt zurück, das den Status (PENDING, COMPLETED, FAILED) und das Ergebnis enthält.

Beispiel:

```
response =  
stub.RequestResult(taskgrid_pb2.RequestResultRequest(task_id=1))  
  
if response.task.status == "COMPLETED":
```

NameService

Der NameService implementiert die NameService-Schnittstellen, die zur Verwaltung von Workern durch den Dispatcher dienen. Er bietet die Möglichkeit der Anmeldung und Abmeldung von Workern anhand derer Adresse und deren Types. Durch die Verwendung des NameService wird eine Abstraktion zwischen Dispatcher und Workern hergestellt, was das dynamische hinzukommen und verschwinden verschiedener worker mit unterschiedlichen funktionen in dem verteilten System ermöglicht.

- RegisterWorker
 - Request: RegisterWorkerRequest(type: string, address: string)
 - Response: RegisterWorkerResponse(success: bool)
 - Beschreibung: Registriert einen Worker mit seinem Typ und seiner Adresse beim NameService.
- LookupWorker
 - Request: LookupWorkerRequest(type: string)
 - Response: LookupWorkerResponse(address: string)
 - Beschreibung: Gibt die Adresse eines verfügbaren Workers für den angegebenen Typ zurück. Die Auswahl erfolgt per Round-Robin-Verfahren.
- DeregisterWorker
 - Request: DeregisterWorkerRequest(address: string)
 - Response: DeregisterWorkerResponse(success: bool)

- Beschreibung: Entfernt einen Worker aus der Liste, basierend auf seiner Adresse.
- GetWorkerStats
 - Request: WorkerStatsRequest()
 - Response: WorkerStatsResponse(worker_counts: map<string, int32>, worker_addresses: repeated string)
 - Beschreibung: Liefert Statistiken über registrierte Worker, einschließlich der Anzahl pro Typ und deren Adressen.

WorkerService

Die Worker implementieren Schnittstellen, die zur Verarbeitung von Aufgaben dienen. Dabei gibt es eine allgemeine Process Task-Schnittstelle, welche je nach Aufgabentyp die worker-interne Funktion zur Bearbeitung des erfragten Tasks aufruft. Des Weiteren bietet er über die RegisterWithNameService Funktion die Möglichkeit der Registrierung beim Namensdienst.

- ProcessTask
 - Request: Task(id: int32, type: string, payload: string, status: string, timestamp_created: int64)
 - Response: ProcessTaskResponse(success: bool, result: string)
 - Beschreibung: Verarbeitet eine Aufgabe basierend auf ihrem Typ und gibt das Ergebnis oder einen Fehler zurück. Unterstützte Aufgabentypen: reverse, sum, hash, upper, lower, wait, length, average, prime. Der Worker führt die spezifische Logik für den Aufgabentyp aus und sendet das Ergebnis an den Dispatcher zurück.

Beispiel:

```
response = worker.ProcessTask(taskgrid_pb2.Task(
```

```
id=1,

type="reverse",

payload="Hello, World!"

))
```

- **RegisterWithNameService**

- Request: RegisterWithNameServiceRequest(type: string, address: string)
- Response: RegisterWithNameServiceResponse(success: bool)
- Beschreibung: Registriert den Worker mit seinem Typ und seiner Adresse beim NameService. Dies ermöglicht dem Dispatcher, den Worker für Aufgaben des entsprechenden Typs zu finden. Die Adresse enthält typischerweise die IP und den Port des Workers (z.B. "worker-reverse:5000").

Beispiel:

```
response =
stub.RegisterWithNameService(taskgrid_pb2.RegisterWithNameServiceRequest(

type="reverse",

address="worker-reverse:5000"

))
```

MonitoringService

Die Monitoring-Komponente implementiert eine Schnittstelle zur Systemüberwachung. Dabei greift sie über GetSystemStats auf den Dispatcher zu.

- **GetSystemStats**

- Request: `SystemStatsRequest()`
- Response: `SystemStatsResponse(active_workers: int32, pending_tasks: int32, avg_processing_time: float, service_connections: map<string, bool>, task_type_stats: map<string, TaskTimingStats>)`
- Beschreibung: Liefert Systemstatistiken, einschließlich aktiver Worker, ausstehender Aufgaben, durchschnittlicher Verarbeitungszeit und Verbindungsstatus.

REST-API (Monitoring)

Die Monitoring-Komponente bietet eine REST-API über Flask, die auf Port 8080 läuft.

- GET /stats
 - Beschreibung: Liefert detaillierte Systemstatistiken, einschließlich aktiver Worker, ausstehender Aufgaben, Verbindungsstatus und Zeitstatistiken pro Aufgabentyp.
 - Response: JSON-Objekt mit den Feldern `active_workers`, `pending_tasks`, `avg_processing_time`, `worker_types`, `service_connections`, `task_type_stats`, `active_worker_list`.

Beispiel:

```
{
  "active_workers": 3,
  "pending_tasks": 2,
  "avg_processing_time": 1.5,
  "worker_types": {"reverse": 1, "sum": 2},
  "service_connections": {"nameservice": true, "dispatcher": true},
  "task_type_stats": {
```

```
"reverse": {"avg_time": 0.5, "max_time": 1.0, "last_time": 0.4,
"recent_times": [0.4, 0.5]}

},

"active_worker_list": [

{"type": "reverse", "address": "worker-reverse:5000", "status": "ACTIVE"}

]

}
```

- GET /workers
 - Beschreibung: Gibt eine Liste der aktiven Worker mit Typ, Adresse und Status zurück.
 - Response: JSON-Array von Objekten mit den Feldern type, address, status.

Beispiel:

```
[

{"type": "reverse", "address": "worker-reverse:5000", "status":
"ACTIVE"},

{"type": "sum", "address": "worker-sum:5001", "status": "ACTIVE"}

]
```

Fazit zur Schnittstellenbeschreibung

Die Schnittstellen des TaskGrid-Systems ermöglichen eine effiziente und skalierbare Kommunikation zwischen den Komponenten. Durch die Verwendung von gRPC wird eine performante und typensichere Kommunikation gewährleistet, während die REST-API der Monitoring-Komponente eine dynamische und skalierbare Möglichkeit zur

Systemüberwachung bietet, welche in Zukunft beispielsweise auf eine Dashboard-Anwendung im Browser erweitert werden kann.

Die klar definierten Schnittstellen erleichtern die Integration und Erweiterung des Systems. Auch wird durch die Verwendung der Codegenerierung der RPC-Funktionen durch eine definierende .proto Datei die Einheitlichkeit der verschiedenen Funktionen gewährleistet, was Konflikte vermeidet.

7 Dokumentation des Buildprozesses

Dieser Abschnitt beschreibt den Buildprozess der Taskgrid-Anwendung, die durch die docker-compose.yml-Datei definiert wird. Der Prozess wird mit Docker Compose gesteuert und erstellt Docker-Images für alle Dienste.

Die docker-compose.yml-Datei definiert mehrere Dienste (z. B. nameservice, dispatcher, verschiedene Worker, monitoring, test-client) und ein Netzwerk (taskgrid). Jeder Dienst hat eine build-Anweisung, die angibt, wie das entsprechende Docker-Image erstellt wird.

Ablauf des Buildprozesses

Der Buildprozess wird durch den Befehl docker-compose build oder docker-compose up --build (siehe Startanleitung) gestartet. Folgende Schritte werden ausgeführt:

1. Parsen der Konfiguration: Docker Compose liest die docker-compose.yml-Datei und identifiziert die Dienste sowie deren Build-Anweisungen.
2. Erstellen der Images:
 - Für jeden Dienst wird die angegebene Dockerfile (z. B. docker/nameservice.Dockerfile oder docker/worker.Dockerfile) im Kontext des aktuellen Verzeichnisses ausgeführt.
 - Build-Argumente (args) wie WORKER_TYPE und PORT werden an die Dockerfile übergeben, um die verschiedenen Dienste wie worker-reverse oder worker-sum zu spezialisieren.

- Images werden nach dem Schema `<projektname>_<dienstname>` benannt (z. B. `taskgrid_nameservice`).

Besonderheiten in der Docker-Compose.yml

- Wiederverwendung der Worker-Dockerfile: Alle einzelnen Worker (`worker-reverse`, `worker-sum`, etc.) verwenden dieselbe Dockerfile (`worker.Dockerfile`), unterscheiden sich jedoch durch `WORKER_TYPE` und `PORT`.
- Abhängigkeiten: Die `depends_on`-Anweisungen stellen sicher, dass Dienste wie `dispatcher` oder `test-client` erst nach ihren Abhängigkeiten (z. B. `nameservice`) gestartet werden.
- Netzwerk: Alle Dienste sind dem `taskgrid`-Netzwerk zugeordnet, das die Kommunikation über DNS-Namen wie `nameservice:50051` ermöglicht.

Der Buildprozess erstellt effizient Docker-Images für alle Dienste, wobei die generische Worker-Dockerfile und Build-Argumente die Erweiterbarkeit und Übersichtlichkeit erhöhen.

8 Startanleitung

1. Bevor das Projekt gestartet werden kann, muss der Docker-Daemon gestartet werden
2. Anschließend muss man im Projektordner in das entsprechende Verzeichnis wechseln (Docker-compose.yml muss sich darin befinden):

./VerteilteSysteme/

3. Die Anwendung kann mit folgendem Befehl gestartet werden:

docker-compose up --build

4. Um das Monitoring aufzurufen und die verschiedenen Prozesse der Anwendung zu überwachen kann man auf folgenden Link gehen:

`http://localhost:8080/stats`

5. Die Anwendung kann mit STRG+C im Terminal beendet werden.
6. Um den Container korrekt herunterzufahren, kann man folgenden Befehl verwenden:

`docker-compose down`

9 Auflistung eingesetzter Technologien

Das TaskGrid+-System verwendet die folgenden Technologien, um eine modulare, containerisierte und verteilte Architektur zu realisieren:

1. Python 3.8+

- Hauptsprache für alle Komponenten (Client, Dispatcher, Worker, Namensdienst, Monitoring).
- Ausgewählt für Einfachheit, umfangreiche Bibliotheken und gRPC-Unterstützung.

2. gRPC

- Framework für performante RPC-Kommunikation zwischen Komponenten.
- Verwendet für Client-zu-Dispatcher, Dispatcher-zu-Worker, Worker-zu-Namensdienst und Monitoring-zu-Dispatcher/Namensdienst.
- Bietet effiziente, typsichere Kommunikation mit Protokollpuffern.

3. Protocol Buffers (taskgrid.proto)

- Definiert die task_t-Struktur und Schnittstellen (z. B. SendTask, LookupWorker).
- Verwendet für strukturierte Datenserialisierung und-deserialisierung.

4. Docker

- Containerplattform zum Verpacken jeder Komponente in isolierte Container.

- Gewährleistet konsistente Umgebungen und vereinfacht die Bereitstellung.

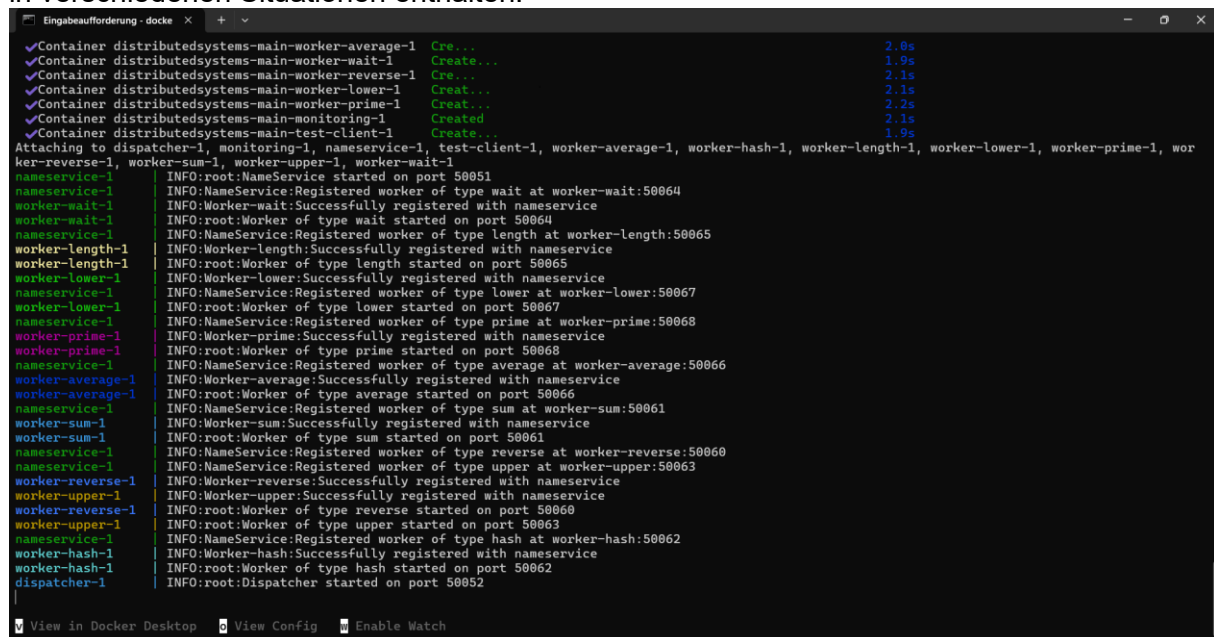
5. Docker Compose

- Orchestriert die Multi-Container-Umgebung und definiert Dienste, Netzwerke und gemeinsame Volumes.
- Vereinfacht das Starten und Skalieren des Systems.

Zusätzlich wurden noch Python Bibliotheken verwendet, siehe hierfür Kapitel 5 „Beschreibung der Verwendeten Externen Bibliothek und deren Funktionalität“

10 Testprotokoll mit Logs

Nachfolgend sind mehrere Screenshots angegeben, welche Beispielausgaben des Systems in verschiedenen Situationen enthalten.



```

Eingabeaufforderung - docke x + v
✓Container distributedsystems-main-worker-average-1 Cre... 2.8s
✓Container distributedsystems-main-worker-wait-1 Create... 1.9s
✓Container distributedsystems-main-worker-reverse-1 Create... 2.1s
✓Container distributedsystems-main-worker-lower-1 Create... 2.1s
✓Container distributedsystems-main-worker-prime-1 Create... 2.2s
✓Container distributedsystems-main-monitoring-1 Created... 2.1s
✓Container distributedsystems-main-test-client-1 Create... 1.9s
Attaching to dispatcher-1, monitoring-1, nameservice-1, test-client-1, worker-average-1, worker-hash-1, worker-length-1, worker-lower-1, worker-prime-1, wor
ker-reverse-1, worker-sum-1, worker-upper-1, worker-wait-1
nameservice-1 INFO:root:NameService started on port 50051
nameservice-1 INFO:NameService:Registered worker of type wait at worker-wait:50064
worker-wait-1 INFO:Worker-wait:Successfully registered with nameservice
worker-wait-1 INFO:root:Worker of type wait started on port 50064
nameservice-1 INFO:NameService:Registered worker of type length at worker-length:50065
worker-length-1 INFO:Worker-length:Successfully registered with nameservice
worker-length-1 INFO:root:Worker of type length started on port 50065
worker-lower-1 INFO:Worker-lower:Successfully registered with nameservice
nameservice-1 INFO:NameService:Registered worker of type lower at worker-lower:50067
worker-lower-1 INFO:root:Worker of type lower started on port 50067
nameservice-1 INFO:NameService:Registered worker of type prime at worker-prime:50068
worker-prime-1 INFO:Worker-prime:Successfully registered with nameservice
worker-prime-1 INFO:root:Worker of type prime started on port 50068
nameservice-1 INFO:NameService:Registered worker of type average at worker-average:50066
worker-average-1 INFO:Worker-average:Successfully registered with nameservice
worker-average-1 INFO:root:Worker of type average started on port 50066
nameservice-1 INFO:NameService:Registered worker of type sum at worker-sum:50061
worker-sum-1 INFO:Worker-sum:Successfully registered with nameservice
worker-sum-1 INFO:root:Worker of type sum started on port 50061
nameservice-1 INFO:NameService:Registered worker of type reverse at worker-reverse:50060
worker-reverse-1 INFO:Worker-reverse:Successfully registered with nameservice
worker-reverse-1 INFO:root:Worker of type reverse started on port 50060
worker-upper-1 INFO:NameService:Registered worker of type upper at worker-upper:50063
worker-upper-1 INFO:Worker-upper:Successfully registered with nameservice
worker-upper-1 INFO:root:Worker of type upper started on port 50063
nameservice-1 INFO:NameService:Registered worker of type hash at worker-hash:50062
worker-hash-1 INFO:Worker-hash:Successfully registered with nameservice
worker-hash-1 INFO:root:Worker of type hash started on port 50062
dispatcher-1 INFO:root:Dispatcher started on port 50052
View in Docker Desktop View Config Enable Watch

```

Hier sieht man die Ausgabe des Systems, die man erhält, wenn Worker registriert werden


```

dispatcher-1 INFO:Dispatcher:Received task 1 of type length
test-client-1 2025-06-10 06:26:45,803 - Task sent successfully. Task ID: 1, Type: length, Payload: 5NU8dGW
worker-length-1 INFO:Worker-length:Processing task 1 of type length
dispatcher-1 INFO:Dispatcher:Task 1 completed with status COMPLETED
test-client-1 2025-06-10 06:26:46,812 - Task 1 completed successfully. Result: 7
test-client-1 2025-06-10 06:26:46,812 - TEST SUMMARY [2025-06-10 06:26:46]:
test-client-1 2025-06-10 06:26:46,813 - Task Type: length
test-client-1 2025-06-10 06:26:46,813 - Input: 5NU8dGW
test-client-1 2025-06-10 06:26:46,813 - Output: 7
test-client-1 2025-06-10 06:26:46,814 - -----
monitoring-1 INFO:werkzeug:172.20.0.1 - - [10/Jun/2025 06:26:47] "GET /stats HTTP/1.1" 200 -
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 2 of type average
test-client-1 2025-06-10 06:27:16,816 - Task sent successfully. Task ID: 2, Type: average, Payload: [87, 4, 44, 97, 2, 3, 26, 4, 49]
worker-average-1 INFO:Worker-average:Processing task 2 of type average
dispatcher-1 INFO:Dispatcher:Task 2 completed with status COMPLETED
test-client-1 2025-06-10 06:27:17,824 - Task 2 completed successfully. Result: 35.111111111111114
test-client-1 2025-06-10 06:27:17,825 - TEST SUMMARY [2025-06-10 06:27:17]:
test-client-1 2025-06-10 06:27:17,826 - Task Type: average
test-client-1 2025-06-10 06:27:17,826 - Input: [87, 4, 44, 97, 2, 3, 26, 4, 49]
test-client-1 2025-06-10 06:27:17,826 - Output: 35.111111111111114
test-client-1 2025-06-10 06:27:17,827 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 3 of type reverse
test-client-1 2025-06-10 06:27:47,838 - Task sent successfully. Task ID: 3, Type: reverse, Payload: x6x83V4kx0d
worker-reverse-1 INFO:Worker-reverse:Processing task 3 of type reverse
dispatcher-1 INFO:Dispatcher:Task 3 completed with status COMPLETED
test-client-1 2025-06-10 06:27:48,847 - Task 3 completed successfully. Result: d0xk4V38x6x
test-client-1 2025-06-10 06:27:48,848 - TEST SUMMARY [2025-06-10 06:27:48]:
test-client-1 2025-06-10 06:27:48,848 - Task Type: reverse
test-client-1 2025-06-10 06:27:48,848 - Input: x6x83V4kx0d

```

Hier sieht man die Ausgabe des Systems für die Tasks: length, average und reverse

```

dispatcher-1 INFO:Dispatcher:Received task 5 of type lower
test-client-1 2025-06-10 06:28:49,893 - Task sent successfully. Task ID: 5, Type: lower, Payload: X0unBQ0eEXZ
worker-lower-1 INFO:Worker-lower:Processing task 5 of type lower
dispatcher-1 INFO:Dispatcher:Task 5 completed with status COMPLETED
test-client-1 2025-06-10 06:28:50,915 - Task 5 completed successfully. Result: x0unbq0eezx
test-client-1 2025-06-10 06:28:50,916 - TEST SUMMARY [2025-06-10 06:28:50]:
test-client-1 2025-06-10 06:28:50,916 - Task Type: lower
test-client-1 2025-06-10 06:28:50,916 - Input: X0unBQ0eEXZ
test-client-1 2025-06-10 06:28:50,916 - Output: x0unbq0eezx
test-client-1 2025-06-10 06:28:50,917 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 6 of type sum
test-client-1 2025-06-10 06:29:20,920 - Task sent successfully. Task ID: 6, Type: sum, Payload: [83, 25, 19, 60, 43, 82]
worker-sum-1 INFO:Worker-sum:Processing task 6 of type sum
dispatcher-1 INFO:Dispatcher:Task 6 completed with status COMPLETED
test-client-1 2025-06-10 06:29:21,935 - Task 6 completed successfully. Result: 312.0
test-client-1 2025-06-10 06:29:21,937 - TEST SUMMARY [2025-06-10 06:29:21]:
test-client-1 2025-06-10 06:29:21,939 - Task Type: sum
test-client-1 2025-06-10 06:29:21,940 - Input: [83, 25, 19, 60, 43, 82]
test-client-1 2025-06-10 06:29:21,941 - Output: 312.0
test-client-1 2025-06-10 06:29:21,941 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 7 of type lower
test-client-1 2025-06-10 06:29:51,959 - Task sent successfully. Task ID: 7, Type: lower, Payload: CcCI879DEmWwF
worker-lower-1 INFO:Worker-lower:Processing task 7 of type lower
dispatcher-1 INFO:Dispatcher:Task 7 completed with status COMPLETED
test-client-1 2025-06-10 06:29:52,985 - Task 7 completed successfully. Result: cccI879demwff
test-client-1 2025-06-10 06:29:52,986 - TEST SUMMARY [2025-06-10 06:29:52]:
test-client-1 2025-06-10 06:29:52,986 - Task Type: lower
test-client-1 2025-06-10 06:29:52,986 - Input: CcCI879DEmWwF
test-client-1 2025-06-10 06:29:52,986 - Output: cccI879demwff
test-client-1 2025-06-10 06:29:52,987 - -----

```

Hier sieht man die Ausgabe des Systems für die Tasks: lower und sum

```

dispatcher-1 INFO:Dispatcher:Received task 3 of type reverse
test-client-1 2025-06-10 06:27:47,838 - Task sent successfully. Task ID: 3, Type: reverse, Payload: x6x83V4kx0d
worker-reverse-1 INFO:Worker-reverse:Processing task 3 of type reverse
dispatcher-1 INFO:Dispatcher:Task 3 completed with status COMPLETED
test-client-1 2025-06-10 06:27:48,847 - Task 3 completed successfully. Result: d0xk4V38x6x
test-client-1 2025-06-10 06:27:48,848 - TEST SUMMARY [2025-06-10 06:27:48]:
test-client-1 2025-06-10 06:27:48,848 - Task Type: reverse
test-client-1 2025-06-10 06:27:48,848 - Input: x6x83V4kx0d
test-client-1 2025-06-10 06:27:48,848 - Output: d0xk4V38x6x
monitoring-1 INFO:MonitoringService:Active workers: 9
test-client-1 2025-06-10 06:27:48,849 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 4 of type average
test-client-1 2025-06-10 06:28:18,875 - Task sent successfully. Task ID: 4, Type: average, Payload: [92, 90, 81, 90, 43, 50, 93, 22, 18]
worker-average-1 INFO:Worker-average:Processing task 4 of type average
dispatcher-1 INFO:Dispatcher:Task 4 completed with status COMPLETED
test-client-1 2025-06-10 06:28:19,888 - Task 4 completed successfully. Result: 64.33333333333333
test-client-1 2025-06-10 06:28:19,889 - TEST SUMMARY [2025-06-10 06:28:19]:
test-client-1 2025-06-10 06:28:19,889 - Task Type: average
test-client-1 2025-06-10 06:28:19,890 - Input: [92, 90, 81, 90, 43, 50, 93, 22, 18]
test-client-1 2025-06-10 06:28:19,890 - Output: 64.33333333333333
test-client-1 2025-06-10 06:28:19,899 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 5 of type lower
test-client-1 2025-06-10 06:28:49,893 - Task sent successfully. Task ID: 5, Type: lower, Payload: X0unbQ0eEXZ
worker-lower-1 INFO:Worker-lower:Processing task 5 of type lower
dispatcher-1 INFO:Dispatcher:Task 5 completed with status COMPLETED
test-client-1 2025-06-10 06:28:50,915 - Task 5 completed successfully. Result: x0unbq0eezx
test-client-1 2025-06-10 06:28:50,916 - TEST SUMMARY [2025-06-10 06:28:50]:
test-client-1 2025-06-10 06:28:50,916 - Task Type: lower
test-client-1 2025-06-10 06:28:50,916 - Input: X0unbQ0eEXZ
test-client-1 2025-06-10 06:28:50,916 - Output: x0unbq0eezx

```

Hier sieht man die Ausgabe des Systems für die Tasks: reverse, average, lower

```

dispatcher-1 INFO:Dispatcher:Received task 25 of type upper
test-client-1 2025-06-10 06:39:16,965 - Task sent successfully. Task ID: 25, Type: upper, Payload: 1DMj0i14ziNSG8
worker-upper-1 INFO:Worker-upper:Processing task 25 of type upper
dispatcher-1 INFO:Dispatcher:Task 25 completed with status COMPLETED
monitoring-1 INFO:MonitoringService:Active workers: 9
test-client-1 2025-06-10 06:39:17,969 - Task 25 completed successfully. Result: 1DMJ0I14ZINSg8
test-client-1 2025-06-10 06:39:17,969 - TEST SUMMARY [2025-06-10 06:39:17]:
test-client-1 2025-06-10 06:39:17,969 - Task Type: upper
test-client-1 2025-06-10 06:39:17,969 - Input: 1DMj0i14ziNSG8
test-client-1 2025-06-10 06:39:17,969 - Output: 1DMJ0I14ZINSg8
test-client-1 2025-06-10 06:39:17,969 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 26 of type sum
test-client-1 2025-06-10 06:39:47,965 - Task sent successfully. Task ID: 26, Type: sum, Payload: [53, 30, 26, 25, 31, 47]
worker-sum-1 INFO:Worker-sum:Processing task 26 of type sum
dispatcher-1 INFO:Dispatcher:Task 26 completed with status COMPLETED
test-client-1 2025-06-10 06:39:48,973 - Task 26 completed successfully. Result: 212.0
test-client-1 2025-06-10 06:39:48,974 - TEST SUMMARY [2025-06-10 06:39:48]:
test-client-1 2025-06-10 06:39:48,974 - Task Type: sum
test-client-1 2025-06-10 06:39:48,974 - Input: [53, 30, 26, 25, 31, 47]
test-client-1 2025-06-10 06:39:48,974 - Output: 212.0
test-client-1 2025-06-10 06:39:48,975 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 27 of type wait
test-client-1 2025-06-10 06:40:18,981 - Task sent successfully. Task ID: 27, Type: wait, Payload: 2.063008504062399
worker-wait-1 INFO:Worker-wait:Processing task 27 of type wait
dispatcher-1 INFO:Dispatcher:Task 27 completed with status COMPLETED
test-client-1 2025-06-10 06:40:21,998 - Task 27 completed successfully. Result: Waited for 2.063008504062399 seconds
test-client-1 2025-06-10 06:40:21,999 - TEST SUMMARY [2025-06-10 06:40:21]:
test-client-1 2025-06-10 06:40:21,999 - Task Type: wait
test-client-1 2025-06-10 06:40:22,000 - Input: 2.063008504062399
test-client-1 2025-06-10 06:40:22,000 - Output: Waited for 2.063008504062399 seconds

```

Hier sieht man die Ausgabe des Systems für die Tasks: upper, sum und wait

```

Eingabeaufforderung - docke x + v
dispatcher-1 INFO:Dispatcher:Received task 16 of type wait
test-client-1 2025-06-10 06:34:31,655 - Task sent successfully. Task ID: 16, Type: wait, Payload: 2.2142046515613973
worker-wait-1 INFO:Worker-wait:Processing task 16 of type wait
dispatcher-1 INFO:Dispatcher:Task 16 completed with status COMPLETED
monitoring-1 INFO:MonitoringService:Active workers: 9
test-client-1 2025-06-10 06:34:34,668 - Task 16 completed successfully. Result: Waited for 2.2142046515613973 seconds
test-client-1 2025-06-10 06:34:35,705 - TEST SUMMARY [2025-06-10 06:34:35]:
test-client-1 2025-06-10 06:34:35,705 - Task Type: wait
test-client-1 2025-06-10 06:34:35,706 - Input: 2.2142046515613973
test-client-1 2025-06-10 06:34:35,706 - Output: Waited for 2.2142046515613973 seconds
test-client-1 2025-06-10 06:34:35,706 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 17 of type prime
test-client-1 2025-06-10 06:35:05,707 - Task sent successfully. Task ID: 17, Type: prime, Payload: 60
worker-prime-1 INFO:Worker-prime:Processing task 17 of type prime
dispatcher-1 INFO:Dispatcher:Task 17 completed with status COMPLETED
monitoring-1 INFO:MonitoringService:Active workers: 9
test-client-1 2025-06-10 06:35:06,715 - Task 17 completed successfully. Result: False
test-client-1 2025-06-10 06:35:06,716 - TEST SUMMARY [2025-06-10 06:35:06]:
test-client-1 2025-06-10 06:35:06,716 - Task Type: prime
test-client-1 2025-06-10 06:35:06,716 - Input: 60
test-client-1 2025-06-10 06:35:06,716 - Output: False
test-client-1 2025-06-10 06:35:06,716 - -----
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
monitoring-1 INFO:MonitoringService:Active workers: 9
dispatcher-1 INFO:Dispatcher:Received task 18 of type length
test-client-1 2025-06-10 06:35:36,816 - Task sent successfully. Task ID: 18, Type: length, Payload: 81xqJzLzpM463ql1pVh
worker-length-1 INFO:Worker-length:Processing task 18 of type length
dispatcher-1 INFO:Dispatcher:Task 18 completed with status COMPLETED
test-client-1 2025-06-10 06:35:37,858 - Task 18 completed successfully. Result: 19
test-client-1 2025-06-10 06:35:37,859 - TEST SUMMARY [2025-06-10 06:35:37]:
test-client-1 2025-06-10 06:35:37,860 - Task Type: length
test-client-1 2025-06-10 06:35:37,861 - Input: 81xqJzLzpM463ql1pVh
test-client-1 2025-06-10 06:35:37,861 - Output: 19
  
```

Hier sieht man die Ausgabe des Systems für die Tasks: wait, prime und length